CEUE203 - OOP25AIML034

PRACTICAL – 9

Aim:

To develop Java multithreading programs to demonstrate race conditions and thread synchronization by incrementing a shared counter, booking seats concurrently, and calculating a shared sum using multiple threads, then compare the incorrect and synchronized results.

Code Screenshot:

```
class Counter {
    int count = 0;

    void incrementUnsafe() {
        count++;
    }

    synchronized void incrementSafe() {
        count++;
    }
}

public class Main {
    public static void main(String[] args)
```

```
    throws InterruptedException {


int threads = 5;

int iterations = 100000;


Counter unsafe = new Counter();

Thread[] t1 = new Thread[threads];


for (int i = 0; i < threads; i++) {

    t1[i] = new Thread(() -> {

        for (int j = 0; j < iterations; j++) {

            unsafe.incrementUnsafe();

        }

    });

    t1[i].start();

}


for (Thread t : t1) {

    t.join();

}


System.out.println("Without synchronization: "
```

```
        + unsafe.count);

Counter safe = new Counter();
Thread[] t2 = new Thread[threads];

for (int i = 0; i < threads; i++) {
    t2[i] = new Thread(() -> {
        for (int j = 0; j < iterations; j++) {
            safe.incrementSafe();
        }
    });
    t2[i].start();
}

for (Thread t : t2) {
    t.join();
}

System.out.println("With synchronization: "
        + safe.count);

System.out.println("Expected count: "
```

```
            + (threads * iterations));
    }
}
```

2.

```
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.atomic.AtomicInteger;

class SeatBooking {
    int seatsLeft = 5;
    AtomicInteger successfulBookings = new AtomicInteger(0);
    CyclicBarrier barrier = new CyclicBarrier(10);

    void bookUnsafe() {
        if (seatsLeft > 0) {
            try {
                // All 10 threads check before any thread books.
                barrier.await();
            } catch (Exception e) {
                Thread.currentThread().interrupt();
                return;
            }
```

```
            seatsLeft--;
            successfulBookings.incrementAndGet();
        }
    }

    synchronized void bookSafe() {
        if (seatsLeft > 0) {
            seatsLeft--;
            successfulBookings.incrementAndGet();
        }
    }
}

public class Main {
    public static void main(String[] args)
            throws InterruptedException {

        SeatBooking unsafe = new SeatBooking();
        Thread[] t1 = new Thread[10];

        for (int i = 0; i < 10; i++) {
```

```
    t1[i] = new Thread(() -> unsafe.bookUnsafe());
    t1[i].start();
}

for (Thread t : t1) {
    t.join();
}

System.out.println("Without synchronization:");
System.out.println("Successful bookings: "
        + unsafe.successfulBookings.get());
System.out.println("Seats left: " + unsafe.seatsLeft);

SeatBooking safe = new SeatBooking();
Thread[] t2 = new Thread[10];

for (int i = 0; i < 10; i++) {
    t2[i] = new Thread(() -> safe.bookSafe());
    t2[i].start();
}

for (Thread t : t2) {
```

```
            t.join();
        }

        System.out.println("\nWith synchronization:");
        System.out.println("Successful bookings: "
                + safe.successfulBookings.get());
        System.out.println("Seats left: " + safe.seatsLeft);
    }
}
```

3.

```
import java.util.concurrent.CyclicBarrier;

class SharedSum {
    long total = 0;
    CyclicBarrier barrier = new CyclicBarrier(4);

    void addUnsafe(long value) {
        long temp = total;

        try {
            // Force all threads to read the same old total.
```

```
            barrier.await();
        } catch (Exception e) {
            Thread.currentThread().interrupt();
            return;
        }

        total = temp + value;
    }

    synchronized void addSafe(long value) {
        total += value;
    }
}

public class Main {

    static final int SIZE = 1000000;
    static final int THREADS = 4;

    static long[] createArray() {
        long[] numbers = new long[SIZE];
```

```
    for (int i = 0; i < SIZE; i++) {
        numbers[i] = i + 1;
    }

    return numbers;
}

static long sumRange(long[] numbers, int start, int end) {
    long sum = 0;

    for (int i = start; i < end; i++) {
        sum += numbers[i];
    }

    return sum;
}

public static void main(String[] args)
        throws InterruptedException {

    long[] numbers = createArray();
    long expected = (long) SIZE * (SIZE + 1) / 2;
```

```
// Method 1: Unsafe shared total
SharedSum unsafe = new SharedSum();
Thread[] t1 = new Thread[THREADS];

long start = System.nanoTime();

for (int i = 0; i < THREADS; i++) {
    final int index = i;
    t1[i] = new Thread(() -> {
        int from = index * (SIZE / THREADS);
        int to = from + SIZE / THREADS;

        long partial = sumRange(numbers, from, to);
        unsafe.addUnsafe(partial);
    });
    t1[i].start();
}

for (Thread t : t1) {
    t.join();
}
```

```
long unsafeTime = System.nanoTime() - start;

System.out.println("Unsafe total: " + unsafe.total);
System.out.println("Expected total: " + expected);

// Method 2: Synchronized shared total
SharedSum safe = new SharedSum();
Thread[] t2 = new Thread[THREADS];

start = System.nanoTime();

for (int i = 0; i < THREADS; i++) {
    final int index = i;
    t2[i] = new Thread(() -> {
        int from = index * (SIZE / THREADS);
        int to = from + SIZE / THREADS;

        long partial = sumRange(numbers, from, to);
        safe.addSafe(partial);
    });
    t2[i].start();
```

```
}

for (Thread t : t2) {
    t.join();
}

long safeTime = System.nanoTime() - start;

System.out.println("\nSynchronized total: " + safe.total);

// Method 3: Thread-local partial sums
long[] partials = new long[THREADS];
Thread[] t3 = new Thread[THREADS];

start = System.nanoTime();

for (int i = 0; i < THREADS; i++) {
    final int index = i;
    t3[i] = new Thread(() -> {
        int from = index * (SIZE / THREADS);
        int to = from + SIZE / THREADS;
```

```
        partials[index] = sumRange(numbers, from, to);
    });
    t3[i].start();
}

for (Thread t : t3) {
    t.join();
}

long localTotal = 0;

for (long partial : partials) {
    localTotal += partial;
}

long localTime = System.nanoTime() - start;

System.out.println("Thread-local total: " + localTotal);
System.out.println("\nExecution times:");
System.out.printf("Unsafe: %.3f ms%n",
        unsafeTime / 1_000_000.0);
System.out.printf("Synchronized: %.3f ms%n",
```

```
            safeTime / 1_000_000.0);
        System.out.printf("Thread-local: %.3f ms%n",
            localTime / 1_000_000.0);
    }
}
```

Output Screenshot:

Output

```
Without synchronization: 366534
With synchronization: 500000
Expected count: 500000
```

Output

```
Without synchronization:
Successful bookings: 10
Seats left: -5

With synchronization:
Successful bookings: 5
Seats left: 0
```

Output

```
Unsafe total: 218750125000
Expected total: 500000500000

Synchronized total: 500000500000
Thread-local total: 500000500000

Execution times:
Unsafe: 16.809 ms
Synchronized: 35.500 ms
Thread-local: 35.680 ms
```